

Analysis of Algorithms

- When we have two algorithms that solve the exact same problem, how can we compare them?
- We need to evaluate execution time and memory usage.

Time execution depends on:

- The Input of the algorithm: size and characteristics.
- Logic and structure of the algorithm
- Code quality generated by the compiler
- Speed of the host machine (hardware)

We need a way to compare algorithms what is independent on hardware

- Sumatory of i to the third power N times.

```
int sum(int n)
{
    int partialSum = 0;

    for(int i=1; i <= n; i++)
        partialSum += i * i * i;

    return partialSum;
}
```

Asymptotic Notation

- Temporal complexity: Time required by an algorithm
- Depending on the bounds considered, we can distinguish three types of notations: Big-O (Upper bound), Big-Sigma (Lower bound), Big-Theta (closest approximation to real execution time).

Big O Notation

- Definition: The function $T(n)$ is the number of steps of an algorithm. If there exists positive constants c and n_{sub0} such that $T(n) \leq c \cdot g(n)$ $c = \text{constant}$ whenever $n \geq n_{sub0}$.
- Many upper bouwnnds can be true for the same function but we always report the tightest (best) one.

- If $T(n) = 2n^2$ then $T(n) = O(n^2)$

Typical complexities

- $O(1)$ Constant
- $O(\log n)$ Logarithmic
- $O(n)$ Linear
- $O(n \log n)$ Log-linear
- $O(n^2)$ Quadratic
- $O(n^3)$ Cubic
- $O(n^c)$ Polynomial ($c = \text{constant}$)
- $O(m^n)$ Exponential ($m = \text{constant}$)
- $O(n!)$ Factorial

Shortcuts for loops

- Linear is when the variable is incremented or decremented by a constant $O(n)$ for example $i++$
- Logarithmic is when the variable is multiplied or divided by a constant $O(\log n)$ for example $i*=2$
- For a nested loop the complexity is multiplying the inner loop and the outer loop $O(n * n)$ or $O(n^2)$.